Verifiable Matrix Maintenance with Fresh Work Guarantees

Chuanliu Liu, Zhirong Yue, Rong Du, Yue Fu, and Yongle Chen

Taiyuan University of Technology
{liuchuanliu0360, 2024002578}@link.tyut.edu.cn
{durong, fuyue, chenyongle}@tyut.edu.cn

***Abstract*—Incremental matrix maintenance reuses previous products, but retained state also changes the fresh computation needed to obtain acceptance from an outsourced service. We study finite-field updates with arbitrary paid history and homogeneous bilinear programs fixed before fresh factors are disclosed. An output-space argument conditioned on the factor's rank yields an explicit agreement profile in the online gate budget. It strictly improves the corresponding uncapped range bound at every positive budget below nr. A resource recursion composes acceptance bounds under adaptive pathwise budgets, including incorrectly accepted intermediate states. Conditional linear escape extends the threshold guarantee to nonuniform inputs. Balancing minimizes both homogeneous bounds; an exact allocation algorithm handles heterogeneous rounds. A 2×2 six-product Strassen witness attains the single-round bound over every finite field. Exact-field implementations measure the full pipeline, including candidate materialization and checks. Dense updates and Covertype views exercise structured reuse, while randomized categorical reports instantiate the nonuniform guarantee. The resulting profiles connect encodings, checking decisions, and a specified work target, and the measurements expose the practical boundary between cached state, fresh computation, and verification.**

I. Introduction

Matrix products support analytical views, matrix chains, and repeated feature transformations. Incremental view maintenance preserves these results as records arrive or change. DBToaster develops higher-order maintenance, LINVIEW treats linear-algebra programs, and F-IVM combines factorization with ring-based analytics [1]–[3]. For $C = AB$ and $A' = A + UV^\top$, the established identity

$$C' = C + U(V^\top B)$$

reuses the previous product and exposes the computation associated with the new update.

An outsourced service must also verify returned results and, when it accounts for useful computation, relate acceptance to work on newly disclosed inputs. These requirements interact with reuse. A prover can retain products, intermediate values, and arbitrary auxiliary state, and adapt its later programs and expenditure to earlier requests. A random projection checks a fixed candidate cheaply [4]; bounding accepted computation additionally requires a work model and causal input order.

Unlike ordinary maintenance, this setting fixes the program before the update and the candidate before the checks. Without these boundaries, retained tables can select input-dependent shortcuts or shape candidates after seeing the checking vectors. We therefore separate preprocessing, fresh gates, materialization, and checking, and target a composable acceptance statement for a prescribed update sequence.

We use a fixed invertible $B \in \mathbb{F}_q^{n\times n}$. Before each pair of factors $U, V \in \mathbb{F}_q^{n\times r}$ is disclosed, the prover fixes a homogeneous bilinear straight-line program. Its coefficients may depend on its complete paid history. Work counts products between a linear form in U and a linear form in V. The full candidate is fixed before the verifier samples independent checks. The interface permits arbitrary retained state while separating previous information from fresh inputs. Uniform factors yield an explicit work profile; fresh randomized categorical reports give a concrete nonuniform instance. The prototype realizes this interface with a trusted evaluator for admitted kernels.

The main observation is that a w-gate program outputs into a fixed space of dimension at most w, whereas a fresh factor of rank s leaves the target uniform on a space of dimension ns. Keeping these rank strata until after the probability is truncated at one produces a strict improvement over the unconditional range bound. This refinement complements general bilinear agreement results [5] and the exact-computation tensor threshold.

- **Work-dependent acceptance.** We derive the rank-conditioned profile and compose it through a resource recursion under adaptive pathwise budgets. The proof includes incorrect accepted references (Lemma 3, Theorem 3).
- **Checking decisions and sharpness.** We prove optimal balancing for homogeneous rounds and exact pseudo-polynomial allocation for heterogeneous threshold bounds. A 2×2 six-product Strassen construction attains the general single-round bound (Theorems 4–6).
- **Maintained analytical state.** Matched exact-field implementations measure reuse, candidate materialization, and checking on dense updates and real-data views. Randomized per-attribute reports connect the nonuniform guarantee to a choice of encoding and verification budget (Sections V–VI).

II. Related Work

a) Incremental analytics: Classical materialized-view work separates delta maintenance from full refresh and studies the maintenance of warehouse summaries [6], [7]. Continuous-query semantics and differential dataflow extend the same idea to streams and incremental collections [8], [9]. DBToaster, LINVIEW, and F-IVM establish the higher-order, factorized, and algebraic maintenance underlying our interface [1]–[3]. These systems optimize the maintained value; our contribution is a fresh-work acceptance bound that remains valid when a processor retains arbitrary state and the accepted reference may be wrong.

b) Verification and useful work: Freivalds checks products by random projections, while Pantry and Dumas–Zucca address stateful or public matrix verification [4], [10], [11]. Cryptographic verifiable computation establishes non-interactive proofs for general circuits, from the outsourcing model of Gennaro et al. to Pinocchio and succinct zero-knowledge arguments [12]–[14]. These systems trade setup, proof, and cryptographic assumptions for generality. Komargodski and Weinstein study useful-work proofs for arbitrary matrix multiplication in a preprocessing and hardness framework [15]. Our guarantee is information-theoretic within the stated online bilinear class. Alman and FFLAS-FFPACK also show why finite- field bit complexity and arithmetic-gate counts must be reported separately [16], [17]. Verifiable query systems in data engineering provide a complementary systems perspective [18].

c) Coded and fast matrix computation: Polynomial and Lagrange coded computing use algebraic encodings to tolerate stragglers and, in some formulations, protect inputs or outputs [19], [20]. Their decoding guarantees concern distributed availability and privacy, whereas our gate budget is charged on fresh factors and composed with output checks. Classical fast matrix algorithms and tensor-rank bounds remain relevant to the admitted bilinear class [21]–[23]. Cohen and Moshkovitz relate agreement to slice, geometric, and analytic rank; geometric rank of rectangular multiplication is now explicit [5], [24]. Finite-field rank counts supply our profile weights [25]; the six-product witness connects to defective tensor products and support rank [26], [27].

d) Allocation and application mechanisms: Our allocation results specialize ordered-sum threshold representations and separable concave allocation [28], [29]. The application follows randomized-response and locally private frequency estimation mechanisms [30]–[33]. These mechanisms determine the conditional escape parameters and connect encoding accuracy to work-target feasibility; they do not replace the separate integrity constraint imposed by checking.

III. Model and Stateful Protocol

We study an application that maintains a matrix product as new low-rank updates arrive. The application retains its earlier result; the processor may retain arbitrary paid state. A round has two ordering requirements: the bilinear program is selected before the new factors, and the complete candidate is fixed before the verifier's random vectors (Figure 1).

A. Inputs, state, and computation

Fix a finite field $\mathbb{F}_q$, integers $n \geq r \geq 2$, an initial $A_0 \in \mathbb{F}_q^{n\times n}$, and $B \in \mathrm{GL}_n(\mathbb{F}_q)$. The authenticated initial product is $C_0 = A_0B$. The intended trajectory of accepted updates satisfies, in round j,

$$U_j, V_j \xleftarrow{\$} \mathbb{F}_q^{n\times r}, \qquad A_j = A_{j-1} + U_jV_j^\top, \\ C_j = A_jB = C_{j-1} + U_j(V_j^\top B). \tag{1}$$

Uniformity includes rank-deficient factors. Let $\mathcal{H}_j^-$ be the complete classical history just before disclosure: earlier factors, outputs, checking vectors, all suppliers' state, the selected program, its gate allocation, and the coins used to select them. Conditional on every such history, U_j and V_j remain independent and uniform. Section IV-A gives a conditional nonuniform extension.

Preprocessing can depend arbitrarily on A_0, B, and earlier inputs. Its actual cost P is charged, and finite aggregate state S includes all advice, tables, circuits, and caches held by all suppliers. The theorem places no numerical bound on P or S: it constrains the representation and timing of the computation that uses the new factors.

Definition 1 (Admitted online program)**.** *Conditional on $\mathcal{H}_j^-$, the increment is one fixed homogeneous bilinear program*

$$F_j(U,V) = \sum_{a=1}^{w_j} Z_{j,a}L_{j,a}(U)R_{j,a}(V), \quad Z_{j,a} \in \mathbb{F}_q^{n\times n}. \tag{2}$$

The linear forms, output coefficients, topology, and gate count w_j are fixed before the current factors arrive. A gate multiplies one linear form in each fresh factor group. The count includes every supplier's bilinear gates contributing to the round; unused gates may have zero output coefficients.

The fixed transform $V^\top B$ is linear in fresh inputs. Its field operations, state access, additions, communication, and output materialization retain their actual execution costs, but are not the variable-by-variable gates counted by w_j. Exact fast bilinear algorithms are admitted. Input-indexed program selection, branching on current inputs, and nonlinear encodings of those inputs are distinct computational models [16]. Paid state may select the next program before disclosure; an answer computed from the current input by another supplier must be charged as current computation.

B. Candidate and verification

The verifier stores a reference $\widehat{C}_{j-1}$, initially C_0. The candidate is $\widetilde{C}_j = \widehat{C}_{j-1} + F_j(U_j, V_j)$. Its entire actual matrix is fixed before the verifier draws $t_j \geq 1$ independent uniform vectors $z_{j,h} \in \mathbb{F}_q^n$, conditional on the history, factors, and candidate. The verifier tests

$$(\widetilde{C}_j - \widehat{C}_{j-1})z_{j,h} = U_j\big(V_j^\top(Bz_{j,h})\big), \quad h \in [t_j]. \tag{3}$$

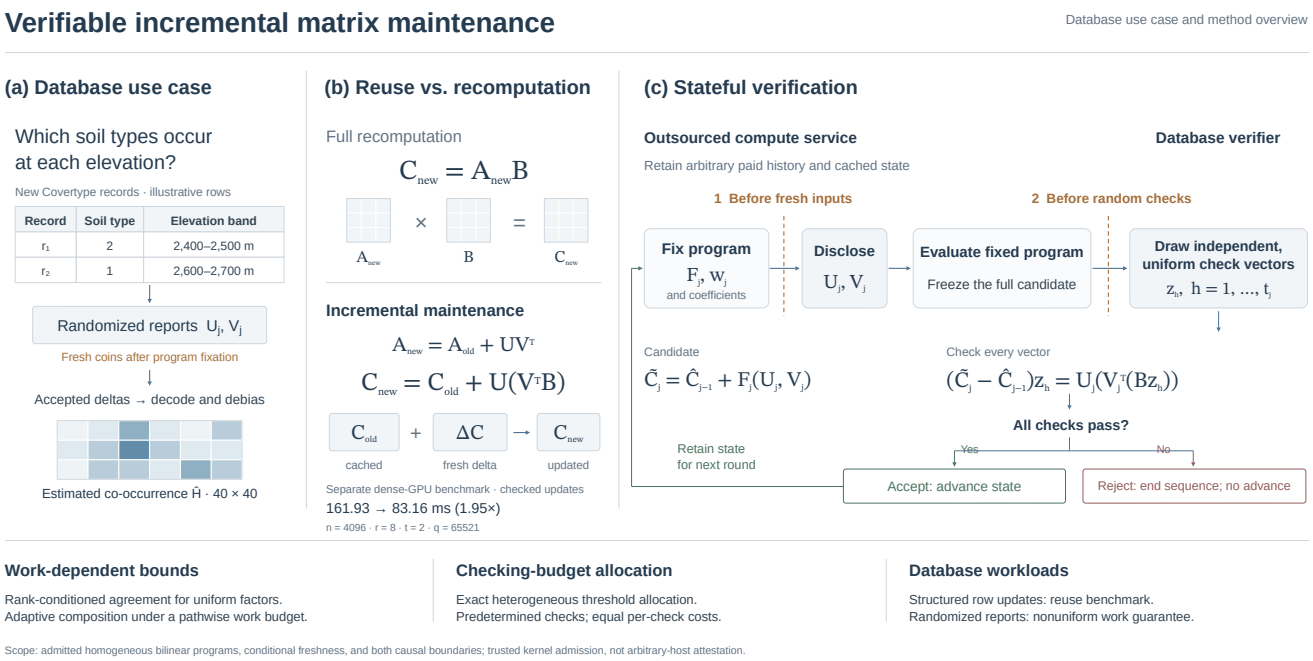


Fig. 1. Overview of the maintained analytical view, cached update, and the two causal boundaries used by verification.

Acceptance commits A_j and advances $\widehat{C}_j$ to the candidate. Rejection commits neither update and ends the prescribed sequence. Recovery starts a new sequence from an authenticated, consistent checkpoint, with its own work budget and acceptance event. Failed candidates and transcripts may remain in paid state; freshness in a restarted sequence is conditioned on this full history. The accepted reference alone serves as the previous-product reference. A fully received matrix satisfies the candidate boundary; a cryptographic representation must bind a complete matrix and account for its binding error.

We consider k prescribed attempts with a *pathwise* budget $\sum_{j=1}^{k} w_j \le W$. Choices may adapt to the complete observed history. Aborting before round k fails the all-execute-and-accept event; selecting successes from unbounded retries is a different event.

A concrete matrix view accumulates cross-products of paired feature streams. A batch of r paired n-dimensional records forms the columns of U_j, V_j; the view maintains $A_j = A_0 + \sum_{i \le j} U_i V_i^\top$, and fixed B transforms its right-hand features. The uniform model describes independently sampled finite-field records. Independent coordinates with bounded conditional maximum atoms give the nonuniform instance of Corollary 1. These are assumptions on the incoming records, rather than consequences of requesting the product. Section VI also measures a structured row-replacement view through the same update interface.

IV. Fresh Computation and Acceptance Bounds

We first establish the tensor threshold and a history-conditioned single-round bound. Corollary 2 refines this bound as a function of work, and Theorem 3 composes the resulting profiles across adaptive rounds. Theorems 5 and 6 then allocate a fixed budget of checking vectors.

Let $\mathcal{R}_q(n, r, n)$ be the tensor rank of $(U, Y) \mapsto UY$, with $U \in \mathbb{F}_q^{n \times r}$ and $Y \in \mathbb{F}_q^{r \times n}$, and define

$$R = \mathcal{R}_q(n, r, n), \qquad \theta_q = 2/q - 1/q^2 = 1 - (1 - 1/q)^2. \quad (4)$$

Lemma 1 (Fresh-input tensor)**.** *The map $T_B(U, V) = U(V^\top B)$ is linearly isomorphic to the $\langle n, r, n \rangle$ multiplication tensor. Its tensor rank is $R \ge n^2$. For any fixed pre-input state, an admitted program exact on all inputs uses at least R bilinear gates.*

Proof. The input change $\Phi_B(V) = V^\top B$ is invertible, with inverse $\Phi_B^{-1}(Y) = (YB^{-1})^\top$. Tensor rank is invariant under this change, while each summand of (2) is a simple input–input–output tensor. Conditioning on $\mathcal{H}_j^-$ makes all coefficients constants relative to the new inputs, however they were prepared, so the program tensor has rank at most w_j.

For the explicit lower bound, output flattening sends a functional λ to $(U, Y) \mapsto \lambda(UY)$. If this form vanishes, the choices $U = e_i e_1^\top$, $Y = e_1 e_\ell^\top$ imply $\lambda(E_{i\ell}) = 0$ for every i, ℓ. The flattening is thus injective on an n^2-dimensional space. A simple tensor has flattening rank at most one, proving $R \ge n^2$. These sparse inputs are algebraic witnesses at the same factor width r.

Over a finite field, equality of bilinear functions determines their tensors: evaluation on pairs of input basis vectors recovers every coefficient. Hence $w_j < R$ implies $F_j - T_B$ is a nonzero bilinear map as a tensor and as a function. □

Fast rectangular algorithms can reduce the direct $n^2 r$ gate count, while every exact admitted program still needs at least

$R \geq n^2$ gates. This argument counts computation, separately from writing the n^2 output entries.

Theorem 1 (One-round exactness and acceptance)**.** *Fix a history $\mathcal{H}_j^-$ and a program with $w_j < R$. Its increment agrees with T_B with probability at most θ_q. With t checking vectors its acceptance probability is at most*

$$a(t) = \theta_q + (1 - \theta_q)q^{-t}. \tag{5}$$

If the preceding reference is genuine, the increment-agreement event is exactly $\widetilde{C}_j = C_j$.

Proof. A nonzero coordinate of $F_j - T_B$ is $p(u, v) = u^\top M v$, where $u = \mathrm{vec}(U)$, $v = \mathrm{vec}(V)$, and $\mathrm{rank}(M) = s \geq 1$. Uniformity gives

$$\mathbb{P}[p(u, v) \neq 0 \mid \mathcal{H}_j^-] = (1-q^{-s})(1-q^{-1}) \geq (1-q^{-1})^2. \tag{6}$$

Indeed, $M^\top u \neq 0$ with probability $1 - q^{-s}$; conditional on such a u, its inner product with independent uniform v is uniform in $\mathbb{F}_q$. Full increment agreement implies $p = 0$.

Write $D = F_j(U_j, V_j) - T_B(U_j, V_j)$. The candidate and the checker use the same previous reference, so (3) tests $Dz = 0$ even if that reference is wrong. After candidate fixation, nonzero D passes t independent vectors with probability $q^{-t\,\mathrm{rank}(D)} \leq q^{-t}$ [4]. For increment-agreement probability $p_j \leq \theta_q$, total acceptance is at most $p_j + (1 - p_j)q^{-t} \leq a(t)$. □

A. History-conditioned nonuniform factors

Let $u_j = \mathrm{vec}(U_j)$ and $v_j = \mathrm{vec}(V_j)$. For each round, suppose predetermined parameters $\lambda_{U,j}, \lambda_{V,j} \in [0, 1]$ satisfy, on every reachable pre-input history h,

$$\mathbb{P}[a^\top u_j \neq 0 \mid h] \geq \lambda_{U,j}, \quad a \neq 0,$$
$$\mathbb{P}[d^\top v_j \neq 0 \mid U_j = u, h] \geq \lambda_{V,j}, \quad d \neq 0,$$

for every supported u. These are uniform lower bounds across histories and nonzero linear forms. The conditioning of the second bound on U_j allows dependencies between the factors without treating them as independent. The invertible change $V \mapsto V^\top B$ preserves the lower bound for nonzero linear forms.

Lemma 2 (Conditional bilinear escape)**.** *Every nonzero bilinear form $u_j^\top M v_j$ is nonzero with conditional probability at least $\lambda_{U,j}\lambda_{V,j}$.*

Proof. Choose a nonzero column a of M. The event $a^\top u_j \neq 0$ has probability at least $\lambda_{U,j}$ and implies $M^\top u_j \neq 0$. For every such realized u_j, the second bound applies to the linear form with coefficient $M^\top u_j$, giving conditional probability at least $\lambda_{V,j}$. Averaging over these u_j proves the claim. □

Corollary 1 (Independent-coordinate escape)**.** *Let $\alpha_j, \beta_j \in [0, 1]$. Suppose coordinates of U_j are conditionally independent given h, with maximum atom at most α_j, and coordinates of V_j are conditionally independent given (U_j, h), with maximum atom at most β_j. The escape conditions hold with $\lambda_{U,j} = 1-\alpha_j$ and $\lambda_{V,j} = 1-\beta_j$. For a fixed discrepancy form whose coefficient matrix has rank s, its nonzero probability is at least $(1 - \alpha_j^s)(1 - \beta_j)$.*

Proof. In a nonzero linear form, condition on all but one coordinate with a nonzero coefficient. At most one value of that coordinate makes the form zero, with conditional probability at most its maximum atom. For the rank-sensitive statement, select s pivot coordinates in the rank-s system $M^\top u_j = 0$. Conditional on the remaining coordinates, the pivot values are uniquely determined. Their conditional independence bounds this event by α_j^s. On its complement, apply the single-form bound $1 - \beta_j$ conditional on U_j. □

Uniform coordinates give escape $1 - 1/q$; independent Bernoulli(p) coordinates give at least $\min\{p, 1 - p\}$. Conversely, a factor supported on a linear hyperplane can have zero escape. Structured data streams must establish the stated conditional parameters before these probability guarantees apply; measuring an update operator's runtime does not supply those distributional parameters.

B. Heterogeneous adaptive acceptance

Theorem 2 (Stateful aggregate bound)**.** *For the prescribed k rounds, assume the conditional escape bounds of Lemma 2 hold on every reachable pre-input history, including histories with incorrectly accepted updates. Let $t_j \geq 1$ be predetermined and the check vectors follow* (3)*. The escape parameters are predetermined history-uniform bounds as defined in Section IV-A, not history-varying realizations. Suppose $\sum_j w_j \leq W$ on every path. Put $b = \min\{k, \lfloor W/R \rfloor\}$, $\theta_j = 1 - \lambda_{U,j}\lambda_{V,j}$ and $a_j = \theta_j + (1 - \theta_j)q^{-t_j}$. Then*

$$\mathbb{P}[\textit{all } k \textit{ rounds execute and accept}] \leq \max_{\substack{E \subseteq [k] \\ |E| \leq b}} \prod_{j \notin E} a_j. \tag{7}$$

The programs and allocations may adapt to the complete observed history; an abort or rejection fails this event. The same maximum with θ_j in place of a_j bounds the probability that all k rounds execute and their increments are exact; an abort before round k fails this event.

Proof. Write $T_j = U_j V_j^\top B$ and $D_j = F_j(U_j, V_j) - T_j$. On every realized pre-input history, F_j is fixed. If $w_j < R$, Lemma 1 gives a nonzero bilinear discrepancy coordinate, so Lemma 2 gives $\mathbb{P}[D_j = 0 \mid \mathcal{H}_j^-] \leq \theta_j$. This argument does not require a genuine stored reference. The check subtracts the same reference used to form the candidate, and is exactly $D_j z_{j,h} = 0$. Given the factors and frozen candidate, a nonzero D_j passes all independent vectors with probability $q^{-t_j\,\mathrm{rank}(D_j)} \leq q^{-t_j}$; thus a cheap round accepts with conditional probability at most a_j. An expensive round ($w_j \geq R$) accepts with probability at most one.

At most b rounds are expensive on any path. For s remaining expensive allowances, set $V_{k+1}(s) = 1$ and recursively define $V_j(s) = \max\{a_j V_{j+1}(s), V_{j+1}(s-1)\}$, omitting the second term when $s = 0$. Backward induction over complete histories bounds the remaining all-accept probability by $V_j(s)$: the cheap branch uses its conditional acceptance bound and the

worst continuation after each accepted history; the expensive branch spends one allowance. Randomized choices are averaged, and aborts cannot increase success. Expanding the recurrence proves (7); replacing acceptance by $D_j = 0$ proves the increment statement. No independence across rounds is needed. □

a) Conditional protocol composition: An implementation transfers the theorem's bound when it enforces the selected program, its evaluation relation, the shared previous-state reference, the pathwise allocation, and the two causal boundaries. If its execution can be coupled to the ideal process except with joint probability ϵ over the prescribed sequence, the all-accept bound increases by at most ϵ. This follows by a union bound without conditioning freshness on absence of failure. Our local candidate digest fixes bytes before check generation; it is not a public attestation of program execution or physical work. With $E_j = \widehat{C}_j - C_j$ and $D_j = F_j(U_j, V_j) - U_j V_j^\top B$, accepted states obey $E_j = E_{j-1} + D_j$: an exact increment preserves an existing state error, and later errors can cancel.

C. Uniform factors and singular fixed operands

For uniform factors and $t_j = t$, put $b = \min\{k, \lfloor W/R \rfloor\}$. Theorem 2 gives

$$\mathbb{P}[\text{all } k \text{ attempts execute and accept}] \le a(t)^{k-b}. \tag{8}$$

This holds for adaptive programs and allocations under the pathwise budget. It is one when $b = k$. The work-sensitive profile below retains the individual gate counts instead of only this expensive-round count.

If $\operatorname{rank}(B) = s$, write $B = P \operatorname{diag}(I_s, 0) Q$ with invertible P, Q. The input change $V' = P^\top V$ and output change $C \mapsto CQ^{-1}$ give $(U, V') \mapsto [U(V'_{1:s,:})^\top \; 0]$. Restriction and projection recover $\langle n, r, s \rangle$, and embedding gives the reverse reduction. The fresh tensor rank is therefore $\mathcal{R}_q(n, r, s) \ge ns$, by the same output flattening. At $B = 0$ the increment is zero. The main workload uses invertible B.

D. A Work-Sensitive Finite-Field Bound

Retain $n \ge r \ge 2$, invertible B, and conditionally independent uniform $U, V \in \mathbb{F}_q^{n \times r}$. Complete pre-input history fixes the program and gate allocation, including arbitrary paid state; the complete candidate precedes the checking vectors.

Lemma 3 (Output-span agreement)**.** *For $0 \le s \le r$, define*

$$\rho_s = q^{-nr} \prod_{i=0}^{s-1} \frac{(q^n - q^i)(q^r - q^i)}{q^s - q^i}, \tag{9}$$

with empty product one, and put

$$\mu_{n,r,q}(w) = \sum_{s=0}^{r} \rho_s q^{-(ns-w)_+}, \qquad (x)_+ = \max\{x, 0\}. \tag{10}$$

For every reachable pre-input history $\mathcal{H}^-$ and admitted w-gate program F,

$$\Pr[F(U, V) = UV^\top B \mid \mathcal{H}^-] \le \mu_{n,r,q}(w), \tag{11}$$

with equality when $w = 0$.

Proof. The outputs of $F = \sum_{a=1}^{w} Z_a L_a(U) R_a(V)$ belong to the fixed space $Z = \operatorname{span}\{Z_1, \dots, Z_w\}$, of dimension at most w. Given rank-s U, the independent uniform matrix $Y = V^\top B$ makes UY uniform on its ns-dimensional image $\mathcal{L}_U$. Thus

$$\begin{aligned} \Pr[F = UY \mid U, \mathcal{H}^-] &\le \Pr[UY \in Z \mid U, \mathcal{H}^-] \\ &= q^{\dim(Z \cap \mathcal{L}_U) - ns} \le q^{-(ns-w)_+}. \end{aligned} \tag{12}$$

Counting full-rank $n \times s$ and $s \times r$ factorizations and dividing by their $|\mathrm{GL}_s(\mathbb{F}_q)|$ multiplicity gives the rank probabilities (9) [25]. Averaging proves the claim, including zero and rank-deficient inputs. At $w = 0$, $F = 0$, and conditional agreement has probability q^{-ns}. □

a) Comparison with bilinear agreement bounds: Cohen and Moshkovitz establish $\mathrm{AR}(D) \ge (1 - \log_q 2)\,\mathrm{GR}(D)$ and $\mathrm{AR}(D) = -\log_q \Pr[D = 0]$ for uniform bilinear inputs [5, Proposition 4.1 and Fact 4.5]. Rectangular multiplication has geometric rank $g = nr - \lfloor r^2/4 \rfloor$ [24] and slice rank nr even over the algebraic closure, by its semistability [34, Example 24 and Lemma 23].[1] For $D = T - F$, take slice rank over the algebraic closure. Subadditivity gives $\mathrm{SR}(D) \ge nr - w$ and $\mathrm{GR}(D) \ge g - w$. Cohen–Moshkovitz's proof gives $\mathrm{SR}(D) \le 2\,\mathrm{GR}(D)$ over that closure [5, proof of Theorem 3.1]; no descent is needed. Since geometric rank is integral, their finite-field analytic-rank inequality yields

$$\Pr[F = T] \le (2/q)^{\max\{\lceil (nr-w)/2 \rceil,\, g-w,\, 0\}}, \quad q > 2. \tag{13}$$

An elementary comparison also uses the exact zero probability: a uniform-input bilinear map's largest output atom occurs at zero [5, proof of Proposition 5.1]. The fixed range of F has at most q^w points, giving $B_{\mathrm{range}}(w) = \min\{1, q^w \mu(0)\}$. Concavity of $z \mapsto \min\{1, z\}$ gives

$$\mu(w) = \mathbb{E}_s \min\{1, q^{w-ns}\} \le B_{\mathrm{range}}(w). \tag{14}$$

This is strict for every integer $0 < w < nr$: the positive rank-zero mass makes $\mu(w) < q^w \mu(0)$, and positive full-rank mass makes $\mu(w) < 1$. At $w = 0$ the two range bounds coincide exactly. At $n = 16, r = 4, q = 17, w = 56$, $\mu(w) = 1.434 \times 10^{-10}$, $B_{\mathrm{range}}(w) = 1.426 \times 10^{-5}$, and (13) gives 1.916×10^{-4}. The rank-conditioned argument also covers $q = 2$, where the geometric comparison's coefficient vanishes. Below a certified tensor-rank threshold, all three bounds can also use the common cap $2/q - 1/q^2$; checking then applies $p \mapsto p + (1-p)q^{-t}$.

Choose a certified integer lower bound $1 \le R_0 \le R = \mathcal{R}_q(n, r, n)$; Lemma 1 permits $R_0 = n^2$, without asserting

[1] If a slice decomposition uses dimensions a_i in modes of dimensions m_i with $\sum_i a_i < \min_i m_i$, assign weights $1 - a_i/m_i$ to its subspaces and $-a_i/m_i$ to their complements. After clearing denominators, each factor has total weight zero, while every supported tensor term has positive weight $\ge 1 - \sum_i a_i/m_i$. This contradicts semistability. The reverse bound follows by slicing a smallest mode.

equality to R. Write $\mu(w) = \mu_{n,r,q}(w)$, $\vartheta = 2/q - 1/q^2$, and define

$$\overline{p}(w) = \begin{cases} \min\{\vartheta, \mu(w)\}, & 0 \le w < R_0, \\ 1, & w \ge R_0, \end{cases} \tag{15}$$

and, for predetermined t_j,

$$\mathsf{Acc}_j(w) = \overline{p}(w) + (1 - \overline{p}(w))q^{-t_j}. \tag{16}$$

Corollary 2 (A conditional acceptance profile)**.** *At every reachable pre-input history, exact-increment and acceptance probabilities are at most $\overline{p}(w)$ and $\mathsf{Acc}_j(w)$, respectively, even with an incorrect previous reference.*

Proof. For $w < R_0 \le R$, a nonzero bilinear coordinate of the discrepancy vanishes with probability at most ϑ by Theorem 1. Combining Lemma 3 gives (15); for $w \ge R_0$ the bound is trivial. The candidate and checker use the same old reference, leaving discrepancy $D = F(U,V) - UV^\top B$ regardless of prior errors. A fixed nonzero D passes all checks with probability $q^{-t_j \operatorname{rank} D} \le q^{-t_j}$. Conditioning on $D = 0$ proves (16). □

Increment and complete-product agreement coincide only with a genuine reference. Acceptance can be strictly smaller even at $w = 0$ when the nonzero discrepancy has rank above one. Transforms, state access, materialization, and checking retain their separate costs.

E. Adaptive Rounds Under a Pathwise Work Budget

Theorem 3 (Aggregate work-profile bound)**.** *Consider k prescribed rounds under the preceding conditional freshness assumptions, predetermined $t_1, \dots, t_k$, and $\sum_j w_j \le W$ on every path. Put $\overline{W} = \lfloor W \rfloor$ and $G_{k+1}(z) = 1$ for integer $z \ge 0$. Define backwards*

$$G_j(z) = \max_{\substack{w \in \mathbb{Z}_{\ge 0} \\ w \le z}} \mathsf{Acc}_j(w) G_{j+1}(z - w). \tag{17}$$

Then

$$\begin{aligned} &\Pr[\textit{all } k \textit{ rounds execute and accept}] \le G_1(\overline{W}) \\ &= \max_{\substack{\mathbf{w} \in \mathbb{Z}_{\ge 0}^k \\ \sum_j w_j \le \overline{W}}} \prod_{j=1}^{k} \mathsf{Acc}_j(w_j). \end{aligned} \tag{18}$$

Replacing $\mathsf{Acc}_j(w)$ by $\overline{p}(w)$ bounds the event that all k rounds execute with exact increments.

Proof. At a pre-input history with remaining budget z and selected allocation w, Corollary 2 bounds acceptance by $\mathsf{Acc}_j(w)$. Every accepted continuation has budget at most $z - w$; backward induction bounds its success by $G_{j+1}(z-w)$ uniformly over complete histories, including incorrect references. Maximizing over w proves (17); randomized choices are mixtures. Aborts and rejections fail the event. Expanding the recurrence proves (18); replacing acceptance by exactness proves the final statement. Cross-round independence is unnecessary. □

a) Finite actions and the threshold comparison: Since $\mu(w) = 1$ for $w \ge nr$, the actions reduce to

$$\mathcal{A} = \{0, \dots, \min(nr, R_0 - 1)\} \cup \{R_0\}, \tag{19}$$

intersected with the remaining budget. If $nr < R_0$, retain nr as the least-cost representative of the $\overline{p}(w) = \vartheta$ plateau through $R_0 - 1$; R_0 represents all larger actions. These replacements preserve profile values at no greater cost. The bound is one for $\overline{W} \ge kR_0$. For the default $R_0 = n^2$, improvement over the threshold profile requires $w < nr$, or normalized work $w/R_0 < r/n$.

With acceptance coefficients precomputed and $M = |\mathcal{A}| \le \min(nr, R_0 - 1) + 2$, value computation takes $O(k(\overline{W}+1)M)$ arithmetic operations/comparisons and $O(\overline{W} + 1)$ auxiliary dynamic-programming (DP) storage. Recovering an allocation stores one maximizing action per state, requiring $O(k(\overline{W} + 1))$ additional entries. These are arithmetic/comparison-model counts. Replacing $\overline{p}(w)$ below R_0 by ϑ recovers the threshold bound: free below-threshold rounds and at most $\min\{k, \lfloor W/R_0 \rfloor\}$ threshold-reaching rounds. The profile strengthens it pointwise.

b) An analytical example: For $n = 4, r = 2, q = 2, k = 4, W = 8, t_j = 1, R_0 = 16$, the 256 matrices U have rank counts $(1, 45, 210)$. Hence

$$\mu(w) = \frac{1 + 45\,2^{-(4-w)_+} + 210\,2^{-(8-w)_+}}{256}. \tag{20}$$

Here $\mu(0) = 593/32768$, $\mu(7) = 151/256 < \vartheta = 3/4$, and $\mu(8) = 1$, so action 8 is essential. A maximizing allocation is $(8, 0, 0, 0)$ up to permutation, giving

$$\begin{aligned} G_1(8) &= \frac{7}{8}\left(\frac{33361}{65536}\right)^3 \approx 0.115421, \\ \text{threshold bound} &= \left(\frac{7}{8}\right)^4 \approx 0.586182. \end{aligned} \tag{21}$$

Indeed, $\log \mathsf{Acc}(w) = \log(A + B2^w)$ is convex on $[0, 4]$ for positive A, B, concentrating work at the endpoints. At most one allocation exceeds four; comparing $(4,4), (5,3), (6,2), (7,1), (8,0)$ with the other two allocations zero establishes the maximum.

Figure 2 compares these analytical bounds and the checking allocations below.

F. Single-Round Sharpness

Karppa and Kaski's six-product Strassen–Winograd variant omits an output monomial [26, Sec. 1.1]. A witness from Strassen's identity [23] attains our one-round bound.

Theorem 4 (Sharpness of the one-round threshold)**.** *For every finite field $\mathbb{F}_q$, $t \ge 1$, $n = r = 2$, and invertible B, an admitted six-gate program has exact-increment probability $\vartheta = 2/q - 1/q^2$ and acceptance probability exactly $\vartheta + (1 - \vartheta)q^{-t}$. Its six gates are fewer than the exact threshold $\mathcal{R}_q(2,2,2) = 7$.*

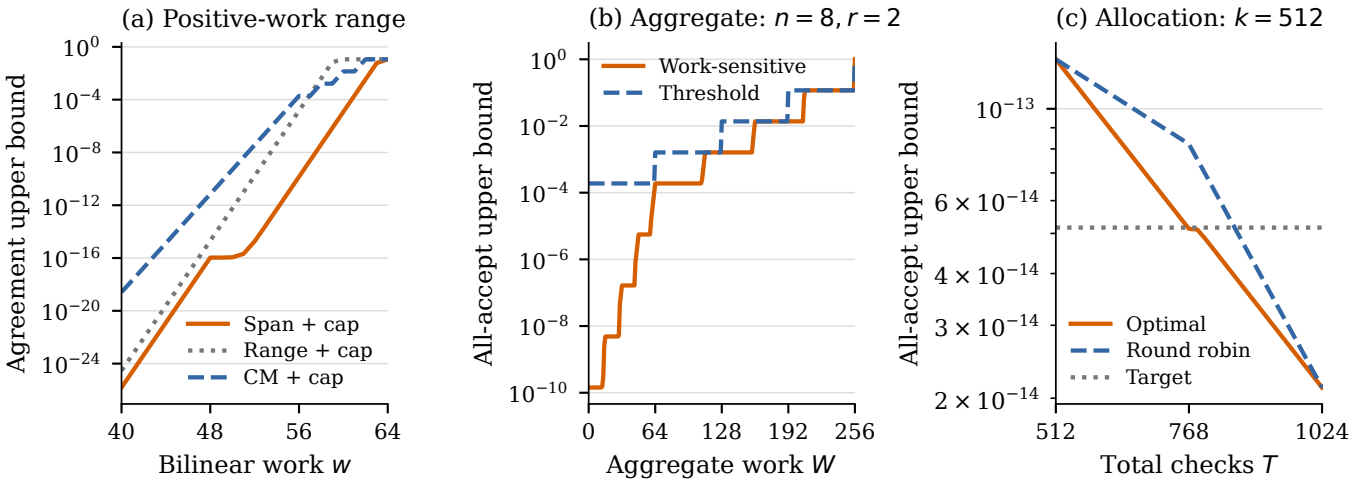


Fig. 2. Analytical agreement, all-accept bounds, and optimal checking allocations under the stated work-budget hypotheses.

Proof. Write $U = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$, $Y = V^\top B = \left[\begin{smallmatrix} e & f \\ g & h \end{smallmatrix}\right]$. Compute

$$\begin{aligned} P_1 &= (a+d)(e+h), & P_2 &= (c+d)e, \\ P_3 &= a(f-h), & P_4 &= d(g-e), \\ P_5 &= (a+b)h, & P_7 &= (b-d)(g+h), \end{aligned}$$

and return

$$F_6 = \begin{bmatrix} P_1 + P_4 - P_5 + P_7 & P_3 + P_5 \\ P_2 + P_4 & P_1 - P_2 + P_3 \end{bmatrix}.$$

The seven-product identity gives

$$F_6 - UY = -E_{22}(c-a)(e+f). \tag{22}$$

The omitted product is never computed. Its two linear forms are independent uniform field elements, including in characteristic two, so their product vanishes with probability $1-(1-1/q)^2 = \vartheta$. Otherwise the discrepancy has rank one and passes t independent checks with probability exactly q^{-t}. The invertible input change preserves the six-gate representation; $\mathcal{R}_q(2,2,2) = 7$ over every field [27]. □

This existing missing-term technique proves single-round sharpness; each witness consumes six gates, with no aggregate tightness claim.

G. Selecting and Distributing Checking Work

Checking counts are predetermined and equally costly; programs may still adapt to prior history.

Corollary 3 (A closed-form checking rule)**.** *Put* $b = \min\{k, \lfloor W/R_0 \rfloor\}$, $d = k - b > 0$, *and* $\alpha(t) = \vartheta + (1-\vartheta)q^{-t}$. *For* $\delta \in (0,1)$, *the uniform threshold certificate* $\alpha(t)^d \le \delta$ *holds exactly when*

$$d \ge \left\lceil \frac{\log \delta}{\log \alpha(t)} \right\rceil.$$

For fixed d, *a finite* t *exists exactly when* $\delta > \vartheta^d$; *the smallest permitted integer is*

$$t_{\min} = \max\left\{1, \left\lceil \log_q \frac{1-\vartheta}{\delta^{1/d} - \vartheta} \right\rceil \right\}. \tag{23}$$

Proof. The equivalent inequality $q^{-t} \le (\delta^{1/d} - \vartheta)/(1-\vartheta)$ requires a positive right side for finite t. Taking logarithms gives both expressions. □

For $d = 0$ the certificate is one. Verify numerical ceilings in the original inequality. The floor ϑ^d concerns this threshold certificate; use (17) for the stronger profile.

Theorem 5 (Balanced checking for identical rounds)**.** *Assume identical* $n, r, q, \overline{p}$ *and equal checking costs. Among integer* $t_j \ge 1$ *with* $\sum_j t_j = T \ge k$, *a balanced allocation minimizes both the threshold and work-profile certificates. Write* $T = k\ell + s$, $0 \le s < k$. *A minimizer has* $k - s$ *entries* ℓ *and* s *entries* $\ell + 1$; *its threshold value, for* $d = k - b$, *is*

$$\alpha(\ell)^{\min\{d, k-s\}}\, \alpha(\ell+1)^{\max\{0, d-k+s\}}. \tag{24}$$

Its work-profile value is given by (17).

Proof. For $c \in [0,1]$, $f_c(x) = \log[c + (1-c)q^{-x}]$ is convex. Thus

$$H(\mathbf{t}) = \max_{\substack{\mathbf{w} \in \mathbb{Z}^k_{\ge 0} \\ \sum_j w_j \le \lfloor W \rfloor}} \sum_j f_{\overline{p}(w_j)}(t_j)$$

is convex; identical profiles and a permutation-invariant budget make it symmetric. The log threshold certificate, a maximum of sums of d functions $f_\vartheta(t_j)$, has the same properties. For integers $u - v \ge 2$, replacing (u, v) by $(u-1, v+1)$ averages the original pair with its swap, with swap weight $1/(u-v)$. Convexity and symmetry cannot increase either objective. Iteration reaches the stated balanced allocation. Its d largest $\alpha(t_j)$ contain $\min\{d, k-s\}$ copies of $\alpha(\ell)$, proving the formula. □

If $s \le b$, the exceptional set contains all strengthened rounds and the threshold value remains $\alpha(\ell)^d$; the profile still uses its recurrence.

H. Heterogeneous Checking Allocation

For the predetermined history-uniform parameters of Theorem 2, put $\vartheta_j = 1 - \lambda_{U,j}\lambda_{V,j}$ and

$$\alpha_j(t) = \vartheta_j + (1-\vartheta_j)q^{-t}, \qquad c_j(t) = -\log \alpha_j(t).$$

For a valid bound $b < k$ on threshold-reaching rounds and $d = k - b$, optimize

$$\Psi(\mathbf{t}) = \max_{\substack{E\subseteq[k]\\ |E|\le b}} \prod_{j\notin E} \alpha_j(t_j) \tag{25}$$

over integer $t_j \ge 1$, $\sum_j t_j = T \ge k$. We specialize ordered-sum thresholds [28] and greedy concave resource allocation [29].

Theorem 6 (An exact heterogeneous checking allocation)**.** *Under equal checking costs and the preceding parameters, form*

$$\mathcal{C} = \{c_j(u) : 1 \le j \le k,\ 1 \le u \le T-k+1\}.$$

For each $\tau \in \mathcal{C}$, start with $t_j = 1$ and repeatedly allocate the remaining $T-k$ checks to a round maximizing

$$\Delta_j = \min\{c_j(t_j+1), \tau\} - \min\{c_j(t_j), \tau\}, \tag{26}$$

breaking ties by fixed index. Select the candidate and threshold maximizing

$$\sum_j \min\{c_j(t_j), \tau\} - b\tau. \tag{27}$$

The selected allocation globally minimizes Ψ at budget T.

Proof. Since $0 < \alpha_j \le 1$, $-\log\Psi$ is the sum of the d smallest $c_j(t_j)$. In nondecreasing order $c_{(i)}$, the ordered-sum identity is

$$\sum_{i=1}^{d} c_{(i)} = \max_{\tau\ge 0} \left\{ \sum_j \min(c_j, \tau) - b\tau \right\}. \tag{28}$$

Its slope is the number of entries above τ minus b; hence $\tau = c_{(d)}$ maximizes it, including ties and $b = 0$. For fixed τ, $\min\{c_j(t), \tau\}$ is nondecreasing and concave. Its discrete gains are nonnegative and nonincreasing. Each feasible allocation takes a prefix of every gain sequence. Taking the largest available gain preserves these prefixes and selects the largest remaining gains, proving fixed-threshold greedy optimality. Every feasible $t_j \le T-k+1$, so an optimal allocation's threshold $c_{(d)}$ belongs to $\mathcal{C}$. Interchanging the two maximizations and enumerating that set therefore attains the global optimum. □

Put $S = T - k$. At most $k(S+1)$ thresholds each require S allocations. A heap uses $O(k(S+1)(k + S\log k))$ reward comparisons in the exact-comparison model, giving pseudo-polynomial complexity. Parameters must remain uniform over all reachable histories, including incorrect acceptances. Optimality concerns (25), not the heterogeneous work-profile minimax objective or actual attack probability.

a) Local descent and integrity requirements: Directly maximizing the one-step decrease of Ψ need not be globally optimal. For $k = 3, b = 2, q = 17, \vartheta_j = 15/16, T = 6$, all available gains are zero along the fixed-smallest-index greedy path from $(1,1,1)$. It returns $(4,1,1)$ with $\Psi = 16/17$, whereas $(2,2,2)$ gives $271/289 < 16/17$.

Conditional checking soundness and a union bound limit the probability of any incorrect accepted increment by $I(\mathbf{t}) = \sum_j q^{-t_j}$, without independence across rounds. For $0 < \vartheta_j \le 1$, define

$$f_\infty = \max_{\substack{E\subseteq[k]\\ |E|\le b}} \prod_{j\notin E} \vartheta_j, \qquad \kappa = \max_j \frac{1-\vartheta_j}{\vartheta_j}.$$

Then

$$f_\infty \le \Psi(\mathbf{t}) \le f_\infty \exp\big(\kappa I(\mathbf{t})\big). \tag{29}$$

Indeed, $\alpha_j(t) = \vartheta_j[1 + (1-\vartheta_j)q^{-t}/\vartheta_j]$; apply $1 + x \le e^x$ to each retained factor and maximize over E.

Theorem 6 also holds with integer minima $t_j \ge L \ge 1$ and $T \ge kL$: start from L, allocate $T - kL$ checks, and enumerate

$$\mathcal{C}_L = \{c_j(u) : j \in [k],\ u = L, \dots, T-(k-1)L\}.$$

The same threshold and decreasing-gain proof gives global optimality among allocations obeying these minima. For $0 < \varepsilon < 1$, choosing $L = \max\{1, \lceil \log_q(k/\varepsilon)\rceil\}$ ensures $I(\mathbf{t}) \le \varepsilon$; this common minimum is a sufficient condition for a general integrity budget.

V. Execution and Resource Costs

A. Platforms and matched operators

The primary server (fy) has four RTX 5090 GPUs. Four DGX Spark hosts (Spark1–Spark4) each provide a GB10 GPU and a 20-core Arm CPU. Python 3.12.3, NumPy 2.5.4, and CuPy 14.2.0 support the word-size backend; CPU BLAS uses four threads. The large-field C++17 kernel is single-threaded. The machines are shared and retain their existing workloads. Independent sweeps use separate devices; each host's long-stream extension runs sequentially.

The incremental operator computes $Y = V^\top B$ and UY, then adds the retained product. Full recomputation forms $A'B$. Both pipelines maintain $A' = A + UV^\top$ when the application retains A. Table I separates this common update from product construction. These direct counts also distinguish the work model's fresh bilinear products from fixed-right transforms, verification, and state access. The theoretical model admits fast bilinear programs.

B. Exact arithmetic and candidate interface

The main backend uses $q = 65521$ with canonical residues. Double-precision (FP64) general matrix-matrix multiplication (GEMM) evaluates each product, followed by conversion to signed 64-bit integers and modular reduction. Every tested inner dimension $d \le 4096$ satisfies

$$d(q-1)^2 \le 4096 \cdot 65520^2 < 2^{44} < 2^{53}.$$

TABLE I
DIRECT MULTIPLICATIONS PER ROUND WHEN BOTH PIPELINES MAINTAIN THE LEFT OPERAND AS PART OF THE VIEW.

Operation	Multiplications
Common $UV^\top$	$n^2 r$
Increment $V^\top B$, then $U(V^\top B)$	$2n^2 r$
Full $A'B$	n^3
One incremental check vector	$2n^2 + 2nr$

All nonnegative integer products and partial sums are exactly representable; subsequent additions are integral. This follows the word-size-field approach of exact linear algebra [17]. The separate $q = 2^{61} - 1$ backend uses 128-bit products, reducing every product and accumulated sum modulo q.

The NumPy/CuPy backends materialize a complete canonical 32-bit candidate, copy it into a read-only host snapshot, and hash its bytes with the round context before generating checks. The $q = 2^{61} - 1$ backend instead delivers a separate immutable 64-bit matrix without a digest. The received copy is checked using operating-system randomness and unbiased rejection sampling. Rejection leaves the reference unchanged and ends the sequence. The digest records byte fixation; the program-class guarantee uses the execution contract of Section IV-B. Retaining A, B, C requires $3n^2$ field words. At $n = 4096$ the candidate alone occupies $64\,\mathrm{MiB}$; the $r = 8$ GPU run's maximum live-array allocation is $1.75\,\mathrm{GB}$, including benchmark reset snapshots.

C. Structured analytical views

With a fixed reference table $Y \in \mathbb{F}_q^{n\times d}$, maintain $A = XY^\top$ and $C = AB$. Replacing b rows of X with difference $D \in \mathbb{F}_q^{b\times d}$ and selector $E \in \mathbb{F}_q^{n\times b}$ gives

$$\Delta A = E(DY^\top), \qquad \Delta C = E((DY^\top)B). \tag{30}$$

Implementations use row gather/scatter. A factorized baseline caches $H = Y^\top B$ and applies $E(DH)$, charging construction of H and its dn retained field words. Setup-inclusive comparisons use each method's actual initialization, including XH where applicable. The deterministic selector has zero linear escape, so this view measures structured reuse. The randomized-report view below also instantiates the nonuniform work guarantee.

Covertype contains $581,012$ records and 54 integer features [35]. The row-replacement study excludes the class label and maps features into $\mathbb{F}_{65521}$. Five disjoint source regions provide initial tables, reference tables, and unrecycled replacement streams, using seed identifiers $101, 211, 307, 401, 503$. Each trial uses n rows per initial table and kb replacement rows. Methods share inputs and rotate execution order across seed/repeat pairs.

D. Randomized categorical views

We maintain a cross-product of reusable attribute reports for Covertype's 40 soil categories and 100-meter elevation bins in $[0, 4000)$ ($n = 40$). Each arriving record supplies one paired report; the aggregate is retained across rounds. Symmetric unary encoding (SUE) reports a true bit with probability $p_j = 1 - \eta_j$ and a false bit with probability $\beta_j = \eta_j \in \{1/4, 1/3\}$. For a fresh-work instance, all report coins are independent and sampled after program fixation. Conditioning on arbitrary source records and history leaves coordinate atoms at most $1 - \eta_j$, hence $\lambda_U = \lambda_V = \eta_j$. Each attribute has privacy budget $\varepsilon_j = 2\log((1-\eta_j)/\eta_j)$; a paired report has $2\varepsilon_j$. Privacy-matched optimized unary encoding (OUE) uses $p_j = 1/2$ and $\beta_j \in \{1/10, 1/5\}$ [33], giving escape $\lambda_U = \lambda_V = \beta_j$.

For a prime $q > 2$, set $r = 2$, $B = I$, and retain one modular state C, where I is the identity matrix, $\mathbf{1}_r$ and $\mathbf{1}$ are all-ones vectors of length r and of the ambient dimension, and lift_q maps a field element to its least nonnegative integer representative. The verifier computes $m_U = U_j\mathbf{1}_r$ and $m_V = V_j\mathbf{1}_r$ from received reports. Correct increments lie in $[0, 2]$, so their unique lift, the decoding increment $\Gamma_j = \mathrm{lift}_q(\widetilde{C}_j - \widehat{C}_{j-1})$, recovers $U_jV_j^\top$ despite cumulative wrap. An increment entry outside $\{0, 1, 2\}$ causes rejection without state advance. On acceptance, advance C and accumulate outside the field:

$$\widehat{H} = \sum_j \frac{\Gamma_j - \beta_j m_U\mathbf{1}^\top - \beta_j\mathbf{1}m_V^\top + r\beta_j^2\mathbf{1}\mathbf{1}^\top}{(p_j - \beta_j)^2}.$$

Correct maintenance gives an unbiased estimator $\widehat{H}$ of the joint category counts by independence of the two report noises. The uniform-factor work profile is unnecessary for this nonuniform application.

Alternate the two parameter types over 512 rounds. Under the global budget $W \le 128n^2$, $R \ge n^2$ permits the conservative allowance $b = 128$ in Theorem 2, using the largest 384 acceptance coefficients. Sorting all 512 rounds gives the global work-certificate floors

$$f_{\mathrm{SUE}} = (625/1024)^{64}, \qquad f_{\mathrm{OUE}} = \big((99/100)^4(24/25)^2\big)^{64}.$$

SUE can certify the target $(31/50)^{64}$; OUE cannot reach it through this threshold certificate.

For the two alternating types, the exact allocator admits a faster specialization of (25). Let $m = 256$, $b = 128$, integer $T \ge 2m$, $S = T - 2m$, and give S_A extra checks to the $\eta = 1/4$ type and $S - S_A$ to the $\eta = 1/3$ type. Convexity and permutation symmetry within each type, as in Theorem 5, ensure a within-type balanced optimum. Because $15/16 > 8/9 + (1/9)q^{-1}$ for $q \ge 17$, the objective always retains all m first-type coefficients and the largest $m - b$ second-type coefficients. Its logarithm is affine between the integer breakpoints

$$\{0, S\} \cup \big(\{hm,\ S - hm,\ S - hm - b : h \in \mathbb{Z}_{\ge 0}\} \cap [0, S]\big).$$

At a multiple of m a balanced level changes; the shifted breakpoints occur when the b excluded second-type entries cross a level. Evaluating these endpoints with exact rational products therefore finds the global optimum at fixed T. Monotonicity in T then permits a search for the smallest total meeting the

target, verified against the optimum at the preceding integer total.

Section VI-D combines the work and integrity requirements at $q = 65521$; the $q = 17$ configuration isolates work-target allocation. Estimation error remains a separate statistical quantity. The implemented fixed bilinear evaluator uses trusted kernel admission and input disclosure; its gate count is a logical resource measure, not hostile-host attestation.

E. Measurement

Matrix-engine latency starts with fresh-input transfer and ends after the accepted state advances. It includes candidate copies, applicable hashing, challenge generation, checks, and necessary device synchronization. Fixtures, resets, and correctness oracles are recorded separately. The unchecked ablation removes only checking from the same candidate interface. Synchronized component diagnostics use separate runs and are not substituted for end-to-end latency.

Each matrix-engine round is compared with a full product on the same backend; small products and selected coordinates are independently checked with Python integers. A mismatch stops that run. Live device arrays, reserved allocator memory, reset snapshots, and process-lifetime RSS are distinguished. The randomized-report service validates typed-plan admission, candidate fixation, reject-without-advance, and per-round integer decoding; a post-stream integer reconstruction checks its complete decoded output. Its verification model and statistical noise are reported separately.

VI. Experimental Evaluation

A. Design and statistical units

The matrix-engine evaluation contains a 73-cell grid and six additional long-stream runs: $176,520$ method-round records, with no full-product discrepancy. Each cell uses five workload seeds and three repeats. Per-update estimates take medians over rounds, repeats, then seeds; speedups use within-seed ratios. Error bars are nominal 95% percentile-bootstrap intervals from $2,000$ resamples of the five seed summaries. Their small-sample resolution and shared-machine timing limit population-level interpretation. Cumulative bands are pointwise and include initialization. The extension reuses the same seeds and retains its separate estimates. The connected study uses twenty same-tape pairs for $q = 17$ replay, twenty fresh source-paired streams per encoding for $q = 65521$ service, and twenty independent utility simulations, displaying the individual units and full view diagnostics in the supplement.

The fixed Covertype extension uses $b = 8, t = 2, k = 256$ at $n \in \{256, 512, 1024\}$ on server GPU2 and $n \in \{512, 1024\}$ on a Spark CPU, plus one synchronized $n = 1024$ diagnostic. Jobs run sequentially within each host. The artifact preserves the full grid, source identities, seed summaries, setup costs, and timing boundaries. The code and lightweight reproducibility materials are available at [2].

[2]https://github.com/fuyuefyu/verifiable-matrix-maintenance

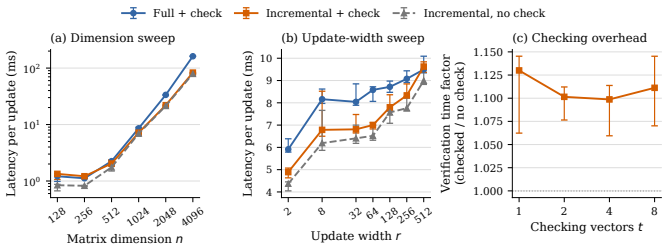


Fig. 3. Engine scaling and checking costs over $\mathbb{F}_{65521}$ with $k = 8$, summarized over paired workload seeds.

B. Strength of the bounds

Figure 2(a) compares the work profile with both the combined slice/geometric-rank and exact-zero-probability range bounds. The positive budget $w = 56$ example improves the latter by approximately 9.95×10^4, locating the gain in rank conditioning rather than zero-work agreement. Composition also changes the certificate: at $n = 4, r = 2, q = 2, k = 4, W = 8, t = 1$, the refined bound is 0.115421 versus the threshold bound 0.586182, even though no round can reach $R_0 = 16$.

Exact enumeration validates the rank formula on 417 matrices, profile allocation on $7,294$ comparisons, and balanced allocation on 360 comparisons. The heterogeneous allocator agrees across $15,147$ configurations and $1,651,995$ candidate comparisons, including $3,360$ cases with tied optima. Its grid covers $q \in \{2, 5, 17\}$, $k = 2, \ldots, 5$, $T = k, \ldots, k + 8$, and $b = 0, \ldots, k - 1$, with uniform, mixed, and endpoint profiles. The single-round witness is checked on $794,884$ input pairs and $19,651,396$ explicit checking vectors over fields of size two, three, and five. These are evaluations of mathematical bounds and procedures, not measured attack probabilities.

C. Complete-update benefits and bottlenecks

At $n = 4096, r = 8, t = 2$, checked GPU maintenance takes 83.162 ms against 161.931 ms for checked recomputation, a paired speedup of $1.947\times$ ($[1.931, 1.985]$). The width sweep reaches $1.968\times$ at $r = 32$ and falls to parity at the largest widths. The complete pipeline's checking factors for $t = 1, 2, 4, 8$ are $1.130, 1.101, 1.099, 1.111$; shared candidate costs prevent a linear extrapolation; the full scaling sweep is shown in Figure 3.

The structured Covertype view gives a second reuse regime, with the full diagnostic retained in the supplement. At $n = 1024, b = 8$, GPU incremental latency is 4.463 ms versus 5.712 ms, a paired $1.279\times$ gain ($[1.227, 1.280]$). The CPU extension reaches $2.098\times$ ($[2.086, 2.112]$). Detailed cache diagnostics, device-specific intervals, and setup-inclusive stage timings are reported in the supplement and remain in the artifact.

D. Encoding and checking choices for randomized views

The categorical view connects report encoding, accuracy, and a fresh-work target. Utility uses twenty independent simulations at four nested prefixes; the performance comparison

uses twenty shared report tapes per policy, while live-service runs draw fresh operating-system coins after each fixed program.

a) Statistical accuracy: The Covertype study evaluates all $1,600$ cells and the twenty cells selected by the true full-data counts. The analytical reference agrees with empirical mean squared error (MSE) within 0.8%. At $N = 581{,}012$, whole-table root MSE is 2.018×10^{-3} for SUE and 1.781×10^{-3} for OUE; their ratio is 1.133 ($[1.123, 1.142]$). Thus SUE's stronger work certificate is accompanied by about 13% higher whole-table root mean MSE, exposing a measurable accuracy–work-guarantee tradeoff. The full variance derivation and all prefix-level cells are in the supplement. For completeness, if $\widehat{H}$ is the released count table, we report $\mathrm{MSE} = n^{-2}\|\widehat{H}/N - H/N\|_F^2$. Writing $\sigma_0^2 = \beta(1-\beta)/(p-\beta)^2$ and $\sigma_1^2 = p(1-p)/(p-\beta)^2$, independence gives the per-record cell-variance sum $[(n-1)\sigma_0^2 + \sigma_1^2 + 1]^2 - 1$. For the equal-parameter strata this predicts $\mathbb{E}[\mathrm{MSE}]_{\mathrm{SUE}} = 47/(20N)$ and $\mathbb{E}[\mathrm{MSE}]_{\mathrm{OUE}} = 1925689/(1036800N)$, which are the curves used in the figure.

b) From a target to measured checking cost: At $q = 17$, the exact allocator (OPT) certifies the minimum of 767 checks for $\delta = (31/50)^{64} = 5.165 \times 10^{-14}$; fixed-prefix round robin (RR) needs 856. Adapting checks to round parameters therefore saves 10.4%. The specialized greedy procedure agrees with exhaustive enumeration on the small validation cases. Complete plan generation costs are 3.473 and 2.743 ms for OPT and RR, and the twenty-session service-plus-planning totals are 983.342 and 997.488 ms, a 1.4% saving for OPT. The adjacent budgets of 766 and 855 checks fail their exact target tests, so the one-check difference is material at this tail probability. Across the frozen validation set, the allocator agrees with exhaustive enumeration on 162 small cases (6,804 allocations) and 56 full-size comparisons; certificate-marginal greedy reaches the same 767-check schedule.

c) Fresh-input service at the integrity setting: At $q = 65521$, the common integrity certificate $I(\mathbf{t}) \le 512/q^2 = 1.193 \times 10^{-7}$ forces at least two checks per round and $T \ge 1024$. Uniform $t = 2$ attains this minimum for SUE and OUE; SUE meets the stated work target, whereas OUE's threshold floor does not. These are analytical certificates under the stated hypotheses, not measured attack probabilities. Table II reports twenty fresh streams per encoding and backend; all 160 replay/live streams complete their 512 rounds and match direct integer reconstruction.

d) Setup-inclusive timing diagnostics: The cached structured path separates one-time factorization from the per-update kernel. At $n = 1024, b = 8$, full product construction takes 1.483 ms, the generic increment 0.0702 ms, and the cached increment 0.0588 ms; candidate freezing and checking add 3.032 and 0.441 ms. Across 256 updates, setup-inclusive medians are 1750.736 ms for recomputation, 1330.161 ms for generic maintenance, and 1325.996 ms for cached factorization. The CPU extension at $n = 1024$ takes 10.234 versus 21.468 ms, while its setup-inclusive totals are 2645.888 and 5520.294 ms; the corresponding cache ratio is $1.020\times$ with interval $[1.010, 1.029]$. GPU speedups at $n = 256$ and 512 are 0.932 and 0.982, so those settings remain within parity once setup is included.

TABLE II
MEDIAN MILLISECONDS PER 512-ROUND FRESH-INPUT $q = 65521$ STREAM ON SPARK-EBAF CPU AND FY RTX 5090 GPU0.

Backend	Encoding	Live time	Active time
CPU	SUE	65.289	51.773
CPU	OUE	64.445	51.592
GPU	SUE	627.120	589.921
GPU	OUE	624.246	588.208

E. Reproducibility boundaries and artifact checks

The reported medians retain the boundary between one-time preparation, candidate formation, and checking. This separation matters for a maintained view: a cached factorization is paid before the first update, whereas a fresh-input certificate is charged on every accepted round. The artifact therefore stores the source identity, workload seed, setup interval, active kernel interval, synchronization point, and acceptance transcript for each cell. A replayed report tape is tagged separately from a fresh operating-system coin stream, so the former checks deterministic reconstruction while the latter checks the stated online randomness model. The complete Covertype diagnostics are shown together in Figure 4.

The dense engine uses $q = 65521$, $n = 4096$, $r = 8$, and $t = 2$ for the primary comparison. Across four GB10 devices, the corresponding dense speedups range from $1.225\times$ to $1.925\times$; devices are reported separately because their memory systems are not pooled. For the structured view, the 54-feature Covertype construction uses $n = 1024$, $b = 8$, and $t = 2$. At smaller dimensions, GPU speedups of 0.932 and 0.982 at $n = 256$ and 512 show why the setup-inclusive totals are retained beside per-update latency. The CPU run uses the same frozen factors and seed order, but is reported as an extension rather than merged into the GPU population.

The integrity-service table also separates planning from execution. The twenty-session totals include serialization, program materialization, fresh source pairing, and the 512-round replay or live stream. Every one of the 160 streams completed all rounds and matched direct integer reconstruction. The repository's dry-run path checks the frozen configuration and source hashes; it does not download data or claim a new scientific execution. These checks make the numerical claims auditable while keeping hardware-dependent reruns explicit.

F. Scope of the empirical claims

The finite-field engine and the randomized categorical view answer different questions. In the engine, every accepted increment is compared with an exact field reconstruction, so the measured latency isolates maintained state, candidate materialization, and checking. In the categorical view, the report is intentionally noisy and the utility target is an estimate of the original count table. We therefore use the former to

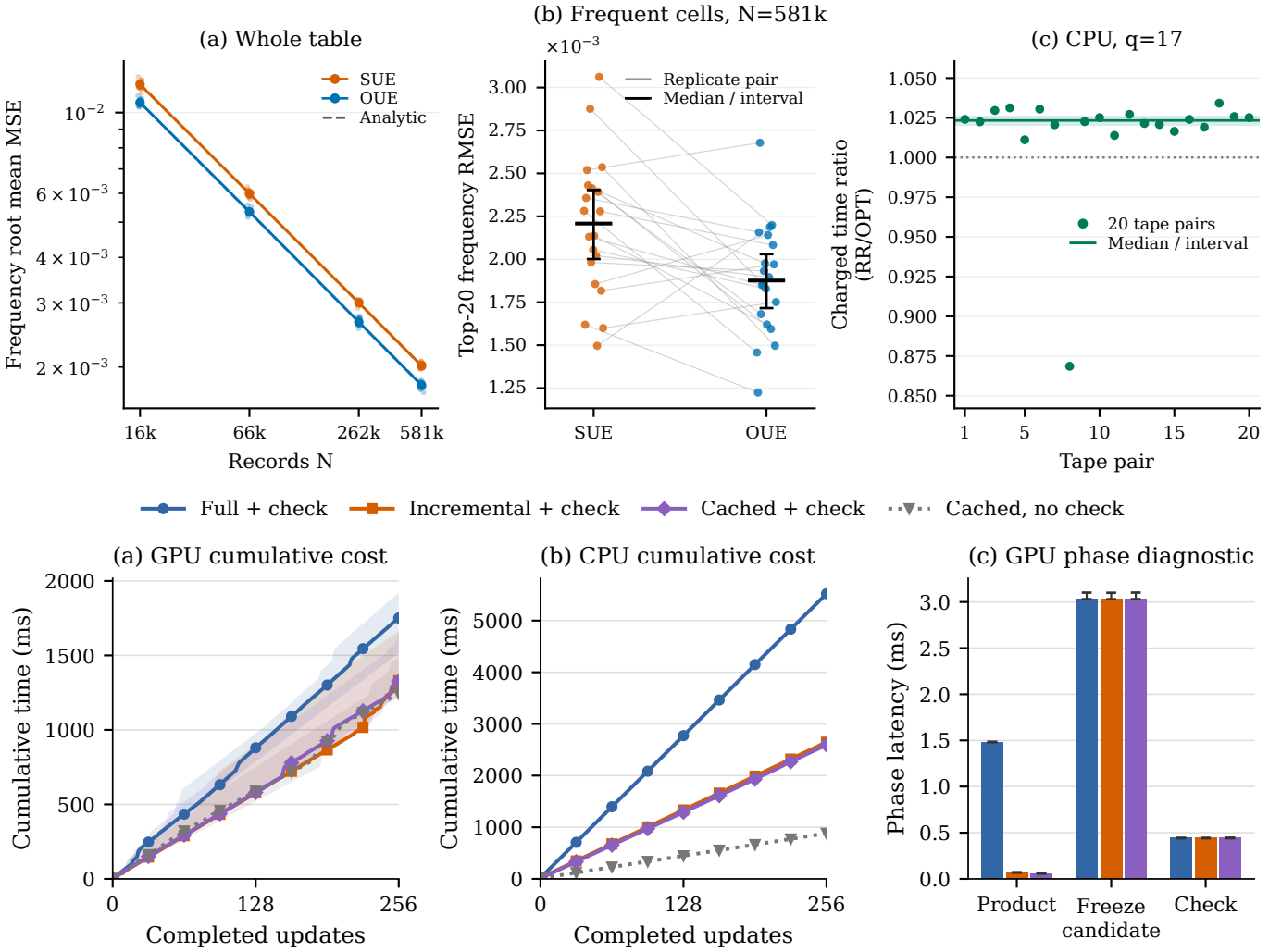


Fig. 4. Covertype diagnostics returned from the supplement: randomized-view accuracy and checking cost (top), and row-replacement cumulative and phase costs for recomputation, incremental maintenance, and cached factorization (bottom).

measure reuse cost and the latter to measure the accuracy–certificate tradeoff; neither experiment is used to infer the other's population-level behavior.

The main grid contains five seeds per cell, and the intervals summarize those seed-level summaries rather than individual method rounds. This preserves the paired design for speedups but leaves limited resolution for distributional claims. The six long-stream runs and the twenty-session service study are reported as frozen extensions with their own denominators. They demonstrate that the implementation completes the prescribed sequence and that the certificate is operational at the stated parameters, while avoiding an unsupported extrapolation to unseen workloads or devices.

The causal ordering is enforced in the experiment harness: the program and its allocation are fixed before fresh factors are disclosed, and the complete candidate is copied before checking vectors are sampled. A digest is retained for dense candidates, while the finite-field service uses an immutable matrix snapshot. These measures make the experiment match the theorem's boundary; they do not claim that an arbitrary untrusted worker will enforce that boundary without an external protocol or trusted execution mechanism.

VII. Discussion and Conclusion

Retained products and arbitrary paid history can coexist with a positive fresh-work guarantee. Rank conditioning and resource recursion give work-dependent acceptance bounds, while exact allocation converts a target into a finite checking decision. Exact-field measurements quantify reuse benefits, and randomized Covertype views instantiate the nonuniform case. The guarantees apply to the stated causal ordering and bilinear class; untrusted general-purpose execution still needs an enforcement mechanism. The empirical results also identify where reuse stops paying: setup costs, candidate freezing, and checking remain visible even when the incremental kernel is small.

Acknowledgment of AI-Assisted Work

AI-assisted tools supported mathematical drafting, code and experiment organization, and language editing. The authors verified the technical content, results, and final presentation and take responsibility for the manuscript.

References

[1] Y. Ahmad, O. Kennedy, C. Koch, and M. Nikolic, "DBToaster: Higher-order delta processing for dynamic, frequently fresh views," *Proceedings of the VLDB Endowment*, vol. 5, no. 10, pp. 968–979, 2012.

[2] M. Nikolic, M. ElSeidy, and C. Koch, "LINVIEW: Incremental view maintenance for complex analytical queries," in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. ACM, 2014, pp. 253–264.

[3] A. Kara, M. Nikolic, D. Olteanu, and H. Zhang, "F-IVM: Analytics over relational databases under updates," *The VLDB Journal*, vol. 33, pp. 903–929, 2024.

[4] R. Freivalds, "Fast probabilistic algorithms," in *Mathematical Foundations of Computer Science 1979*, ser. Lecture Notes in Computer Science, vol. 74. Springer, 1979, pp. 57–69.

[5] A. Cohen and G. Moshkovitz, "Structure vs. randomness for bilinear maps," *Discrete Analysis*, no. 12, pp. 1–21, 2022, an earlier version appeared in STOC 2021. [Online]. Available: https://arxiv.org/abs/2102.04657

[6] A. Gupta and I. S. Mumick, "Maintenance of materialized views: Problems, techniques, and applications," *IEEE Data Engineering Bulletin*, vol. 18, no. 2, pp. 3–18, 1995.

[7] I. S. Mumick, D. Quass, and B. Singh, "Maintenance of data cubes and summary tables in a warehouse," in *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data*. ACM, 1997, pp. 100–111.

[8] A. Arasu, S. Babu, and J. Widom, "The CQL continuous query language: Semantic foundations and query execution," in *Proceedings of the 29th International Conference on Very Large Data Bases*. VLDB Endowment, 2003, pp. 249–260.

[9] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard, "Differential dataflow," in *CIDR 2013*, 2013, pp. 1–12.

[10] B. Braun, A. J. Feldman, Z. Ren, S. Setty, A. J. Blumberg, and M. Walfish, "Verifying computations with state," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. ACM, 2013, pp. 341–357.

[11] J.-G. Dumas and V. Zucca, "Prover efficient public verification of dense or sparse/structured matrix-vector multiplication," in *Information Security and Privacy*, ser. Lecture Notes in Computer Science, vol. 10343. Springer, 2017, pp. 115–134.

[12] R. Gennaro, C. Gentry, and B. Parno, "Non-interactive verifiable computing: Outsourcing computation to untrusted workers," in *Advances in Cryptology – CRYPTO 2010*, ser. Lecture Notes in Computer Science, vol. 6223. Springer, 2010, pp. 465–482.

[13] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 238–252.

[14] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza, "SNARKs for C: Verifying program executions succinctly and in zero knowledge," in *Advances in Cryptology – CRYPTO 2013*, ser. Lecture Notes in Computer Science, vol. 8042. Springer, 2013, pp. 90–108.

[15] I. Komargodski and O. Weinstein, "Proofs of useful work from arbitrary matrix multiplication (invited talk)," in *34th Annual European Symposium on Algorithms (ESA 2026)*, ser. LIPIcs, vol. 388, 2026, pp. 2:1–2:17.

[16] J. Alman, "Faster Walsh–Hadamard transform and matrix multiplication over finite fields using lookup tables," in *Symposium on Simplicity in Algorithms (SOSA)*. Society for Industrial and Applied Mathematics, 2023, pp. 137–144, full version arXiv:2211.04643v1, 9 November 2022; Section 1.1 and Theorem 1.2. [Online]. Available: https://arxiv.org/abs/2211.04643

[17] J.-G. Dumas, P. Giorgi, and C. Pernet, "Dense linear algebra over word-size prime fields: the fflas and ffpack packages," *ACM Transactions on Mathematical Software*, vol. 35, no. 3, pp. 19:1–19:42, 2008.

[18] Z. Yao, T. Li, J. Xin, Y. Li, C. Wang, Z. Wang, D. Srivastava, and C. S. Jensen, "VGQ: Enabling verifiable graph queries on blockchain systems," in *Proc. IEEE International Conference on Data Engineering (ICDE)*, 2025, pp. 3602–3614.

[19] Q. Yu, M. A. Maddah-Ali, and A. S. Avestimehr, "Polynomial codes: An optimal design for high-dimensional coded matrix multiplication," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4403–4413.

[20] ——, "Lagrange coded computing: Optimal design for resiliency, security, and privacy," *IEEE Transactions on Information Theory*, vol. 65, no. 9, pp. 5672–5688, 2019.

[21] D. Coppersmith and S. Winograd, "Matrix multiplication via arithmetic progressions," *Journal of Symbolic Computation*, vol. 9, no. 3, pp. 251–280, 1990.

[22] P. Bürgisser, M. Clausen, and M. A. Shokrollahi, *Algebraic Complexity Theory*, ser. Grundlehren der mathematischen Wissenschaften. Springer, 1997, vol. 315.

[23] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, no. 4, pp. 354–356, 1969.

[24] S. Kopparty, G. Moshkovitz, and J. Zuiddam, "Geometric rank of tensors and subrank of matrix multiplication," *Discrete Analysis*, no. 1, pp. 1–25, 2023, an earlier version appeared in CCC 2020. [Online]. Available: https://arxiv.org/abs/2002.09472

[25] D. Laksov and A. Thorup, "Counting matrices with coordinates in finite fields and of fixed rank," *Mathematica Scandinavica*, vol. 74, pp. 19–33, 1994.

[26] M. Karppa and P. Kaski, "Probabilistic tensors and opportunistic Boolean matrix multiplication," in *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms*. Society for Industrial and Applied Mathematics, 2019, pp. 496–515.

[27] M. Bläser, M. Christandl, and J. Zuiddam, "The border support rank of two-by-two matrix multiplication is seven," 2017, arXiv:1705.09652.

[28] W. Ogryczak and A. Tamir, "Minimizing the sum of the k largest functions in linear time," *Information Processing Letters*, vol. 85, no. 3, pp. 117–122, 2003.

[29] A. Federgruen and H. Groenevelt, "The greedy procedure for resource allocation problems: Necessary and sufficient conditions for optimality," *Operations Research*, vol. 34, no. 6, pp. 909–918, 1986.

[30] S. L. Warner, "Randomized response: A survey technique for eliminating evasive answer bias," *Journal of the American Statistical Association*, vol. 60, no. 309, pp. 63–69, 1965.

[31] U. Erlingsson, V. Pihur, and A. Korolova, "RAPPOR: Randomized aggregatable privacy-preserving ordinal response," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2014, pp. 1054–1067.

[32] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[33] T. Wang, J. Blocki, N. Li, and S. Jha, "Locally differentially private protocols for frequency estimation," in *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, 2017, pp. 729–745. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/wang-tianhao

[34] M. Christandl, V. Lysikov, and J. Zuiddam, "Weighted slice rank and a minimax correspondence to Strassen's spectra," *Journal de Mathématiques Pures et Appliquées*, vol. 172, pp. 299–329, 2023. [Online]. Available: https://arxiv.org/abs/2012.14412

[35] J. A. Blackard, "Covertype," 1998. [Online]. Available: https://archive.ics.uci.edu/dataset/31/covertype